

CHAPTER

5

Cross-Site Request Forgery Defense and Clickjacking

Cross-Site Request Forgery (CSRF) attacks, as the name implies, trick the browser into making unauthorized requests on the victim's behalf, often without the victim's knowledge. This vulnerability is also called "session riding" because it often takes advantage of a legitimate user's existing authenticated session on the vulnerable site. The main defenses we will discuss in this chapter include the synchronizer token pattern, the use of re-authentication or other authentication challenges, referer header verification, as well as stateless defenses such as the double-submit cookie defense. While the synchronizer token pattern is almost always the primary defense needed to stop CSRF, other defenses are sometimes required based on a variety of factors as discussed later in the chapter. But first, let's achieve a deeper understanding of what CSRF is and why it's so terribly dangerous to the security of your web applications.

Cross-Site Request Forgery occurs when several factors are in play at the same time. Here is an example of the most common CSRF attack scenario:

1. The user is currently logged in to a website that is vulnerable to CSRF.

2. The user, without logging off, navigates to another website in some way. This site hosts a CSRF attack against the vulnerable website.

3. The CSRF attack code from the site in step 2 tricks the browser into making fraudulent requests against the vulnerable site in step 1 without the user knowing it. Because the user navigated away to another site and did not log out (a common practice), the fraudulent requests are executed with the same permissions as if the user was browsing the site himself!

Although this scenario is the most common CSRF attack sequence, it is not the only one. CSRF attacks can be stored on the vulnerable website itself, they can be sent via web-based email, or they can be stored on a third-party website. Cross-Site Request Forgery can be used to force a victim to trigger almost any browser action. Actual CSRF attack payloads are only limited by the features of the vulnerable website. Examples of harmful CSRF attacks include submitting a bank transfer, changing a password, changing a router's DNS entries via a web admin console, or fraudulently forcing a user with admin access to give other users elevated privilege. All of these attacks occur without the victim even seeing the action take place! This can be a very dangerous vulnerability in your code!

Cross-Site Request Forgery takes advantage of the fact that when a user is logged in to a vulnerable website, the cookies that are used to maintain their authenticated session will be sent by the browser on any request to that domain—even if the request originated from a different website!

How Does CSRF Work?

Let us consider the example of a form that a website administrator might use to grant a user access to various roles:

Image

When this form is submitted to give user "Moe" access to `Execute`, it will produce an HTTP request like the following:

Image

If user "Larry" wanted to gain `Execute` access for himself, he would simply need to change the value of the `userId` parameter to 1 and resubmit the request, right? The only problem is that pesky `JSESSIONID` cookie. The `JSESSIONID` cookie identifies the valid administrator-level session that is required in order to grant access to other users. It is long, complicated, and difficult to guess by normal means. This is where Cross-Site Request Forgery comes in. If Larry were to construct a form that submits the same parameters to the same URL, all he would need to do is trick an administrator into submitting it:

Image

If Larry tricked an administrator who was logged in to "vulnerable.com" into clicking the Download Free MP3s! submit button, then the session cookie of the administrator would be attached to the request *regardless of what site the request initiated from*. Because HTTP is a stateless protocol, the web application has no way to tell whether the request is actually coming from within the application or from another site. The web application interprets the request as coming from a legitimate authenticated session (which indeed it is), and updates the access levels accordingly. CSRF attacks are typically deployed on high-volume sites, increasing the chance for success.

~~~~~ Image

~~~~~ **NOTE**

~~~~~ *Cookies are crucial for user session management because they are used to track state for most web applications. When cookies are set, the web server includes a domain and path specification for the cookie. Browsers automatically include all cookies in requests to the same domain and path that was specified when the cookie was set.<sup>1</sup> This is crucial in order for web advertising networks, mashup features, and other functionality that many websites depend upon to work. However, this also allows third-party websites to surreptitiously host malicious code that causes users to make forged authenticated requests!*

There are many ways that an attacker could trick an unsuspecting user into submitting the form. Promising something for free is a good example, but with the addition of one line of JavaScript, the victim doesn't need to actually interact with the form at all!

Image

If the vulnerable site allows operations via the HTTP GET method, any HTML that includes a link to another resource can be used to execute a CSRF attack. A classic example is the image tag, as was used in the following real-world CSRF example against a DVD delivery service:

Image

In this example, the vulnerable site allows users to add movies to their list by simply clicking a link (an HTTP GET request). The same URL, when used as the `src` attribute for an image tag, causes the browser to call that link automatically as the page is loading and the movie is added to the victim's list. The extra JavaScript changes the image source attribute after a short delay

and causes the movie to be moved to the top of the user's queue. Loading an image triggers an HTTP GET request and is no different than a user clicking on a normal link.

## Other Real-World CSRF Examples

OSVDB is an independent and open sourced web-based vulnerability database created for the security community. The goal of the project is to provide accurate, detailed, current, and unbiased technical information on security vulnerabilities. The following is a series of interesting CSRF vulnerabilities that have caused a great deal of harm. CSRF is not a theoretical risk; it's an incredibly dangerous risk that we see in the real world on a very regular basis.

Image

## Stored CSRF

The CS in CSRF, *Cross-Site*, alludes to CSRF attacks being hosted on sites other than the victim website. However, if a website is vulnerable to cross-site scripting, as described previously, a stored XSS vulnerability can be used to embed a Cross-Site Request Forgery attack in the victim website itself! Imagine an XSS vulnerability in a social networking site that allows an attacker to post a status update that forces everyone who views it to post the same status update and send a friend request to the attacker? Very likely, anyone running across this attack would already be authenticated, making the impact rather dramatic.<sup>4</sup>

XSS is only one way to embed a CSRF attack on the victim website itself. Any HTML tag that includes a link to another resource, such as `<img>`, `<style>`, or `<iframe>`, can be used by an attacker to force a browser into loading a URL on the victim site. If the aforementioned social networking site added code to prevent attackers from embedding scripts, an attacker could still achieve the same effect by using image tags, or even links (if the user clicks on them).

## CSRF Against Intranet Web Applications

If your website is hosted on your private corporate intranet, is that safe from CSRF? Not necessarily. Although attackers may not be able to directly access your private website they could host an attack on a public website and trick the victim into visiting it. If the attacker knew how the victim website worked (for example, if the attacker was a previous employee or contractor, or if the site uses a common web CMS framework with a known vulnerability), he or she could host a CSRF attack on a third-party website and lure the victim into visiting that website. If the victim was at work or logged in to the company VPN while visiting the malicious website, the CSRF attack could execute against a private intranet site, even if the attacker could not directly access it! This works because the browser attaches session cookies to any request for the site that set them, regardless of where that request is hosted—even forged requests made to private, intranet websites! Ouch!

## CSRF Against Network Application Web Administration Console

One of the most interesting historical CSRF attacks was reported in 2011 by researcher Fabio Assolini. This report described a widespread CSRF campaign against Brazilian home routers that affected at least 4.5 million users.<sup>5</sup> This CSRF attack would allow the attacker to trick a victim into changing the default password of his or her home router to a password that the attacker would know.

Image

The attacker could then use his elevated access to the victims' home routers to change the DNS settings to use malicious DNS servers. This was done to infect the users' machines with malware that ultimately attempted, and sometimes succeeded in, stealing victims' banking credentials. All because of a CSRF vulnerability in the web-based admin console of a home router.

At the time of this writing, this same type of CSRF vulnerability is actively exploiting TP-LINK home routers. As of March 12, 2014, over 300,000 home routers have been affected by a simple CSRF in TP-LINK home routers that will change the admin password to an empty string with a simple HTTP GET CSRF request/attack.<sup>6</sup> How often do you update *your* firmware? A good defense begins by quickly applying security patches. Quickly applying security patches reduces the window of opportunity attackers have to exploit your systems and lowers your risk of exploitation.

## Unauthenticated CSRF Attacks

Even features that do *not* require authentication can be victim to CSRF attacks. One of the most famous but mostly innocent "anonymous" CSRF vulnerabilities was found on Google.com in 2006. This vulnerability, which is no longer active, would allow an attacker to host a simple malicious GET request that would force a user to change his or her default Google language preference. This attack was simple. For example, here is an attack (no longer active) that would force the victim's Google language setting preference to be the Klingon language:

Image

Yes, Google to this day supports Klingon as one of its default languages.<sup>7</sup> The lesson here is simple: *Dujeychugh jagh nIv yItuHQo*.<sup>8</sup> Several types of unauthenticated features can fall victim to CSRF attacks: forcing a user to add an item to a shopping cart, changing default settings on a website, voting in a poll, or conducting DoS attacks. Although authenticated CSRF vulnerabilities are usually much more dangerous than anonymous ones, both must be considered as candidates for CSRF defense.

## How to Combat CSRF

So if you can't trust the browser, and you have no way to know where a request is actually coming from, then how can you possibly protect your applications? Fortunately, combating CSRF is straightforward. Several well-known and well-tested techniques exist to deal with this dangerous attack.

### Synchronizer Token Pattern

The primary means of protecting against Cross-Site Request Forgery is to include a cryptographically random token with form submissions. This Anti-CSRF token is created when a session is established and then included in a hidden field on subsequent form submits. When the request is processed on the server, the application checked that the value submitted with the form matches the value that is stored in the user's session. An attacker creating a form from computed values will not know the value of the token for each user's session, making the CSRF attack impossible as long as the site is not vulnerable to cross-site scripting.

This type of protection can be easily implemented as a JSP taglib:

Image

---

## What Does “Cryptographically Random” Mean?

To be useful in cryptography, a cryptographically random number generator (CRNG) must be unpredictable and it must quickly produce numbers that are hard to guess. Unpredictable means that no amount of knowledge or equipment will allow someone to guess the next number. You could watch the CRNG a billion times and you would still have no way to improve your guess. Every possible number is equally likely. “Hard to guess” means that once the unpredictable number is produced, it should take a long time to guess the number. The longer the number, the more time it takes to guess. Creating long, unpredictable numbers quickly is not easy. We discuss proper random value generation in [Chapter 6](#).

---

On the server, the token can be checked with a method like this:

Image

## Using the Session ID as a CSRF Token

When a user visits a JEE website, the application already generates a cryptographically random token for us: the `JSESSIONID`. One early means of implementing the synchronizer token pattern was to use this session identifier as the Anti-CSRF token:

Image

---

## When Do I Need to Use a Token?

In general, you need to add CSRF protection to any request that performs a “significant transaction.” Any requests that can cause harm if forged should be protected, primarily, with a CSRF token. Not all features need this protection! Simple GET requests that load static resources, such as basic images that drive your website’s look and feel, do not likely need CSRF protection. However, some frameworks support automatic addition of CSRF tokens to all URLs for that website for an even more bullet-proof CSRF protection. So when should you add this defense? Should you protect only sensitive transactions? Should you protect all requests? This is tough to say—it depends on the application. Suffice to say, if the feature is sensitive, regardless of how that request is made (GET or POST), you should add a CSRF token defense scheme to that feature!

---

On the server, you reject any request where the submitted token does not match the session ID. Because an adversary on [attacksite.com](#) cannot read the user’s cookies for [victimsite.com](#), he will have no way to know the proper value for the token and thus the form submission will fail.

So, why don’t you simply use the session identifier as the CSRF token and be done with it? Using the session identifier in this way puts it at risk of exposure in a number of ways:

• Browser cache if the pages do not have the proper cache control headers set

• Cross-site scripting (because it is not just an `HTTPOnly` cookie anymore)

• Applications that log request parameters

• In the URL if tokens are needed for GET requests that perform actions

Depending on the session identifier also means that you cannot simply regenerate the Anti-CSRF token if needed. In general, we feel that using the session ID for CSRF protection should be avoided and that a separate, randomly generated token should be used instead.

## Apache Tomcat 6+ Synchronizer Token Pattern Implementation

Various Java frameworks support the synchronizer token pattern in some fashion. However, implementation differences between platforms can have serious security and functionality implications. It's important to understand how your framework defends against CSRF and whether the built-in option is best for your application. Sometimes the framework solution is acceptable, sometimes it is best to build that defense yourself.

Apache Tomcat provides a CSRF prevention filter available for Java web applications hosted on any version of Tomcat after and including Tomcat 6.<sup>10</sup> This filter implements the synchronizer token pattern for GET or POST requests. The CSRF Prevention Filter protects URLs and POST requests via standard filter mapping. The filter is configurable, with several settings. The `randomClass` setting will let you change the random number generator, which defaults to `java.security.SecureRandom`. This random number generator will be used to create the random CSRF token. The `entryPoint` setting is used for URLs that do not need CSRF protection. When setting up your filter, you will likely force an entire section of your website to enforce CSRF tokens, but you will need an entry page, such as a home page of some kind where tokens are not being verified. There is also a setting, `nonceCacheSize`, which is used to select how many tokens will be cached (for back button and multi-tab support). Each URL protected will have a different token and several page requests opened in several tabs will each have different collections of tokens that will need to be cached. The filter defaults to five cached tokens, but a higher number is often preferred to ensure proper multi-tab support.

The Tomcat CSRF filter will automatically add a token to any URL in the response that is both covered by the filter and is encoded via a call to `HttpServletResponse#encodeRedirectURL(String)` or `HttpServletResponse#encodeURL(String)` as you can see in the URL that follows. If the URLs you send to your user are not encoded via these encoding functions then the token will not be added, yet they will still be enforced server-side and things will start breaking. After encoding, your URLs will look like the following:

Â Â Â Â Â Â Â [http://server/page?org.apache.catalina.filters.CSRF\\_NONCE=31ACB2CA0A9](http://server/page?org.apache.catalina.filters.CSRF_NONCE=31ACB2CA0A9)

This variable name does indeed expose implementation details about the fact that Tomcat 6 or higher is in use and Tomcat's default CSRF filter is in use. There is no simple way to change this variable name other than changing Tomcat code.

Since the Tomcat CSRF filter only adds tokens to URLs but all requests can be configured to require a CSRF token, how does it protect POST form requests? Your `<form src="">` URL also needs to be encoded via `HttpServletResponse#encodeRedirectURL(String)` or `HttpServletResponse#encodeURL(String)`! Unlike other CSRF token injection mechanisms, Tomcat's CSRF filter adds the token to the form action URL, not as a hidden variable.

Because each request is protected by a unique CSRF token with this mechanism, the server needs to cache multiple valid tokens when the user opens several pages of the web application in different tabs. As described previously, five tokens are cached by default and this can be changed via the `nonceCacheSize` parameter. If the `nonceCacheSize` cache is exceeded (in that more than the configured number of tokens is exceeded), then Tomcat's CSRF Prevention Filter can cause things to break when too many tabs are opened. For example, a user could open a

second tab, make `nonceCacheSize` plus one mouse clicks, and now submitting the original tab will fail. Make sure to set this variable high enough!

If you want to provide CSRF coverage to your entire Tomcat web-hosted application, and you are already using the proper format of URL encoding for your application, then you can get CSRF protection very easily by enabling the Tomcat CSRF Protection Filter. If you are not using this URL encoding mechanism and need a large number of URLs to *not* be protected by this filter, then a custom solution might be more appropriate.

## Stateless CSRF Defense

Security is not something you can consider in a vacuum. Sometimes sites have such a large number of users that saving yet another string in the session is something you want to avoid. Some applications forbid the use of the JEE session for performance or load balancing reasons, making it challenging to safely save a randomly generated Anti-CSRF token. Also, stateless web services are common because they promise to enable more concurrent usage with reduced server resources.

One of the more common stateless CSRF defense patterns is the bastard child of the synchronizer token pattern, the double-submit cookie pattern. We prefer the synchronizer token defense described previously, but there are some cases where the double-submit cookie pattern (or other stateless CSRF patterns) actually does make a lot of sense. However, in such situations, you still need to build your CSRF token so it is fully random (best), or a hash of the session ID (not as good). You never want to use the raw session ID for CSRF defense in order to avoid exposing it in any way.

The double-cookie submit CSRF defense works as follows:

1. The user logs in to the website (or some client logs in to a web service).

2. After successful authentication, the server sends the client a cookie containing a large random value and then discards it server-side. This random value is not saved on the server, hence the stateless nature of this defense.

3. The client is now responsible for saving and sending this random value in both the request (as a hidden form field or other request parameter) and in a cookie. The cookie should be at least HTTPOnly because JavaScript code should never need to access this value.

4. The server verifies that the cookie value and request parameter are the same with each request.

How does this work? If the user is logged in to this stateless website or service, a forged request will always include the cookie. However, the attacker should have no way to access or otherwise modify the cookie value, so he cannot add it as a request parameter on a form. Without a request parameter matching the cookie value, the request will be rejected by the server.

Another stateless CSRF defense mechanism is to use the standard synchronizer token pattern described earlier, but to base the value of the token off of the session ID in a safe way instead of using a random value. Because the token will be generated from the session ID, and the session ID is already being saved server-side, the correct CSRF token value can be regenerated on-the-fly for each request without needing to save the token value server-side. So although the server itself may not be stateless, at least the CSRF defense mechanism would be.



The best way to generate a hard-to-guess value from the session ID is to use an HMAC, or an authenticated hash:

Image

Again, the point of this is to provide a mechanism where your CSRF token can be computed and verified on-the-fly without having to save it server-side. An HMAC hash is not easily reversible so there is little risk of exposing the session ID, and the CSRF token cannot be forged as long as the key is kept private on the server.

^ ^ ^ ^ ^ ^ ^ Image

^ ^ ^ ^ ^ ^ ^ **NOTE**

^ ^ ^ ^ ^ ^ ^ *The double-submit cookie defense can be undermined by cookie forcing attacks.<sup>11</sup> The use of Strict Transport Security is critical to battle cookie forcing attacks. We discuss Strict Transport Security in detail in [Chapter 6](#).*

## Defending Against CSRF with the Challenge/Response Pattern

The best way to ensure that a user actually intends to perform an action is to force the user to enter a value known only to him or her, such as a password, or a PIN, in order to complete an action. Because an attacker won't know his victim's password, he won't be able to include that information in the hidden form or link and CSRF is prevented. Presumably, if an attacker already knows his victim's password, he has no need to bother with trying to create a hidden form, getting the user to log in to the victim site, and trying to lure the user to the page containing the CSRF attack.

Requesting that a user reenter their password and/or other credentials is the recommended method to protect very sensitive actions such as changing a user's password or initiating a large money transfer. It also has the additional benefit of preventing an attacker from executing these actions should they compromise the user's session in some other manner (such as by XSS, or simply sitting down at an unlocked workstation).

Another way to ensure that a user actually wants to execute an action is to use a CAPTCHA.<sup>12</sup> Although CAPTCHA can help defeat some automation, in general we see CAPTCHA not as a defensive barrier, but a cost barrier for attackers. However, the usability of secure CAPTCHAs is atrocious at best and we highly recommend avoiding them whenever possible. Re-authentication is, by far, a more usable and more secure challenge-response mechanism.

## HTTP Request Referer Header Verification

Every modern browser sends an HTTP referer header as part of each HTTP request. The HTTP referer contains the URL of the page that the request came from. Another way of saying this is that the HTTP referer hosted the link that was clicked. This is a simple part of the HTTP standard and will look like the following as a header in each request:

Image

Why can't you just use the HTTP referer to protect against CSRF? You could check the referer to make sure the request came from the site you expected instead of a third-party site. Is this a good idea? No, it's not. Remember our First Rule of Web Security: Don't trust anything from the request (or don't trust anything from the browser)! If anything coming from the browser can be spoofed or modified by an attacker, then the referer header is not



sufficient to defend against CSRF. Specifically, some companies strip out the referer from all requests that leave their environment to prevent data leakage. Some web security tools and browser add-ons modify referer headers. Even if the referer header came from your site, the request may still be forged, as is the case of a stored CSRF vulnerability, where the attack is hosted on *your* site, instead of a malicious third-party site.

However, although you cannot depend on the HTTP referer header to *verify* if a request came from the right browser and user, you can use the HTTP referer to *indicate* if the request may be fraudulent. This is a subtle point. Although HTTP referer header verification cannot be used as defensive security control, it can be used for intrusion detection.

If the HTTP referer header contains the right referring URL or is blank, then the request has the *potential* of being legitimate. However, if the referer header is from a third party, and you were expecting the request to originate from your site, then you know for sure the request was fraudulent.

---

## HTTP Verbs

GET and HEAD as “retrieval HTTP verbs only” may have been the intention of Internet architects when the HTTP standard was first created, but it’s certainly not the truth anymore. It’s really up to the site designer if GET only performs retrieval type actions. Of course, there are some practical differences in GET vs. POST data content length limits. Regardless, it’s important to protect any sensitive transactions with proper CSRF defenses.

---

## POST vs. GET

The HTTP protocol suggests that only POST methods should change state and that HTTP GET and other methods should not change the state of the web server in any way: “In particular, the convention has been established that the GET and HEAD methods SHOULD NOT have the significance of taking an action other than retrieval.”<sup>13</sup> In theory then, only POST requests should be dangerous and only POST requests should need CSRF protection. However, some application frameworks choose to add CSRF protection to all URLs and to all form submission/POST requests. This is an acceptable practice, but it does often break bookmarking because a CSRF token bookmarked today should be completely invalid tomorrow.

One way to reduce the attack surface of a potential CSRF attack is to enforce the POST HTTP verb server-side when a POST request is expected. More specifically, when a POST is expected, you should reject any HTTP GET request or any other request with an HTTP verb other than POST! In many apps, both GET and POST requests are accepted unnecessarily when only a POST should be accepted. Enforcing POST or any other HTTP verb server-side is not even *remotely* a complete CSRF defense. As demonstrated, POSTs can be forged just as easily as GETs. But limiting your HTTP verb to POST only does at least reduce the attack surface.

It is also important to note that some Java web frameworks make it extremely difficult to limit which HTTP verb (POST or GET) is appropriate for each request. In these cases, verifying the CSRF token at a central enforcement point is an acceptable practice. Limiting requests to only the appropriate HTTP verb is good practice and does reduce the attack surface but is not absolutely necessary to build a secure web application.

## XSS Defense and CSRF Protection

Any XSS vulnerability can be used to circumvent your CSRF defense. XSS can be used to scrape tokens from pages and replay them. XSS can be used to pop up a fake re-authentication challenge. XSS can access cookies that do not have the HTTPOnly flag. XSS can be used to simulate any request. Therefore, you normally prioritize XSS vulnerabilities as more dangerous than CSRF vulnerabilities. Again, XSS can be used to circumvent any CSRF defense and therefore XSS defense is a critical first step to a complete CSRF defense.

## Clickjacking

Clickjacking,<sup>14</sup> otherwise known as a “User Interface Redress attack,” is a form of attack that tricks a user into clicking on one location when they are really clicking on another location that could cause harm. This attack, similar to CSRF, is often launched from a site other than the site being attacked.

Clickjacking attacks are conducted by

- Image Loading a vulnerable website in a third-party site via an iframe or similar construct

- Image Concealing the presence of the vulnerable website by either making it transparent or covering it with other opaque layers

- Image Tricking the user into thinking she is clicking in some area of the third-party website, when the user is in reality clicking in some area of the vulnerable website

---

## What Do CSRF and Clickjacking Have in Common?

CSRF and clickjacking both abuse the user’s trust in a site other than the one being attacked. These attacks are often hosted on third-party sites with the intention of exploiting a vulnerability on a different site that the user might not even be viewing. Cross-site scripting, on the other hand, abuses the user’s trust in the site they are actually viewing at the time of the attack.

---

## How to Combat Clickjacking

To prevent clickjacking, you must prevent your site from being displayed in a frame by another site. This can be achieved by either preventing your site from being framed, or breaking out of framing sites (framebusting).

### Stop Your Site from Being Framed with Framebusting

The most straightforward defense against clickjacking is to prevent your website from being loaded in a frame in the first place. This is easily achieved by sending an X-Frame-Options HTTP response header. This header instructs compliant browsers that the site is unwilling to be framed by third-party sites. There are three main X-Frame-Options choices:

- Image **X-Frame-Options: DENY** This option blocks *any* site from framing the web page you are trying to protect.

Image **X-Frame-Options: SAMEORIGIN** This option blocks any site from framing the web page you are trying to protect, except for sites of the same domain name and port.

Image **X-Frame-Options: ALLOW-FROM [affiliatesite.com](http://affiliatesite.com)** This option allows only the domain listed after the ALLOW-FROM directive and blocks all others. If you want your site to be frameable by multiple other sites, you must send the X-Frame-Options: ALLOW-FROM header multiple times.

Image

**CAUTION**

*Browser support of X-Frame-Options, and especially X-Frame-Options ALLOW-FROM is mixed. Mozilla maintains a browser-support list for X-Frame-Options at <https://developer.mozilla.org/en-US/docs/HTTP/X-Frame-Options> and Erlend Oftedal supports an excellent X-Frame-Options browser test at <http://erlend.oftedal.no/blog/tools/xframeoptions/>. It should become easier to defend against clickjacking as browser support for X-Frame-Options grows stronger.*

## Break Out of Frames

Though the X-Frame-Options directive is supported by most modern browsers, if your site still needs to support older browsers, that can be achieved by using JavaScript to force a page to be loaded as the top-level window. In constructing such a defense, great care must be taken to ensure that clickjacking is rendered useless, even if the user has JavaScript turned off or otherwise disabled in his browser. The concept is to use CSS styles to hide the entire contents of the page, and then use JavaScript, if enabled, to either break out of a frame or display the hidden content. That way, even if JavaScript is not enabled, clickjacking is impossible because the site contents are hidden.

First, in the document HEAD, add a style element that will hide the document BODY:

Image

Then, use a script that will either break out of a frame if needed, or display the content:

Image

This method, known as the CodeMagi Clickjack Defense<sup>15</sup> is known to work in IE6+, FireFox 3+, and most versions of Chrome, Safari, and Opera.

## Summary

CSRF and clickjacking attacks abuse trust in end-user applications. These risks are even more challenging because attacks of these types are hosted on third-party sites and exploitation at third-party sites is seldom defensible in the end-user application. Even worse, infection vectors for these risks are usually deployed in high-volume websites or spam for maximum impact. However, you can stop both CSRF and clickjacking with proper, secure coding techniques. To stop CSRF, you need to primarily utilize the synchronizer token pattern or the stateless double-submit cookie pattern. To stop clickjacking, you need to master framebreaking primarily via X-Frame-Options headers.

<sup>1</sup> Please note, cookies protected with a Secure flag will only be added to HTTPS requests! Cookies protected with an HTTPOnly flag will not be accessible via JavaScript attacks!

<sup>2</sup> <http://nakedsecurity.sophos.com/2010/09/26/wtf-twitter-goat-viral-message-spreads/>

<sup>3</sup> <http://namb.la/popular/tech.html>

<sup>4</sup> This is exactly how the Samy worm managed to infect more than one million MySpace profiles within 20 hours of its release in 2005.

<sup>5</sup> [https://www.securelist.com/en/blog/208193852/  
The\\_tale\\_of\\_one\\_thousand\\_and\\_one\\_DSL\\_modems](https://www.securelist.com/en/blog/208193852/The_tale_of_one_thousand_and_one_DSL_modems)

<sup>6</sup> <http://arstechnica.com/security/2014/03/hackers-hijack-300000-plus-wireless-routers-make-malicious-changes/>

<sup>7</sup> To experience Google in Klingon, you can safely visit <https://www.google.com/?hl=xx-klingon>.

<sup>8</sup> There is nothing shameful in falling before a superior enemy: <http://hol.kag.org/page/sayings>.

<sup>9</sup> The optional HTTPOnly property of a cookie prevents it from being accessed via JavaScript without changing other cookie behavior.

<sup>10</sup> [https://tomcat.apache.org/tomcat-7.0-doc/config/filter.html#CSRF\\_Prevention\\_Filter](https://tomcat.apache.org/tomcat-7.0-doc/config/filter.html#CSRF_Prevention_Filter)

<sup>11</sup> <http://michael-coates.blogspot.jp/2010/01/cookie-forcing-trust-your-cookies-no.html>

<sup>12</sup> Completely Automated Public Turing test to tell Computers and Humans Apart, <http://en.wikipedia.org/wiki/CAPTCHA>

<sup>13</sup> [www.w3.org/Protocols/rfc2616/rfc2616-sec9.html](http://www.w3.org/Protocols/rfc2616/rfc2616-sec9.html)

<sup>14</sup> The term “clickjacking” was created by Jeremiah Grossman and Robert Hansen in 2008: [www.securityfocus.com/news/11535/](http://www.securityfocus.com/news/11535/)

<sup>15</sup> <https://www.codemagi.com/codemagi-clickjacking-defense>